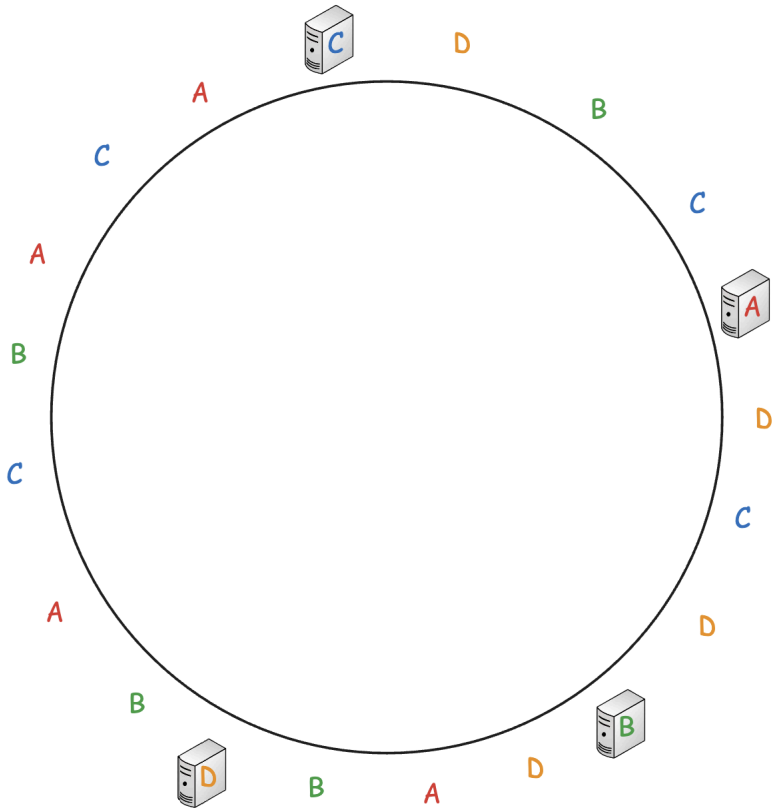
For any suggestions or feedback regarding these notes,
please contact [Pragy Agarwal]

Consistent Hashing (under different settings)

In the first couple of classes we talked about how "Sharding" works.
Sharding always happens via a sharding key.

The routing algorithm (consistent hashing) determines the sharding.
Data is created on the go - if all of Swetha's requests go to server C, then her data will automatically be in server C.

Sharding (without Replication)



Q: Shard vs Server

Every server contains a different shard.

1 Server == 1 Shard
This means that each server has a different set of data.

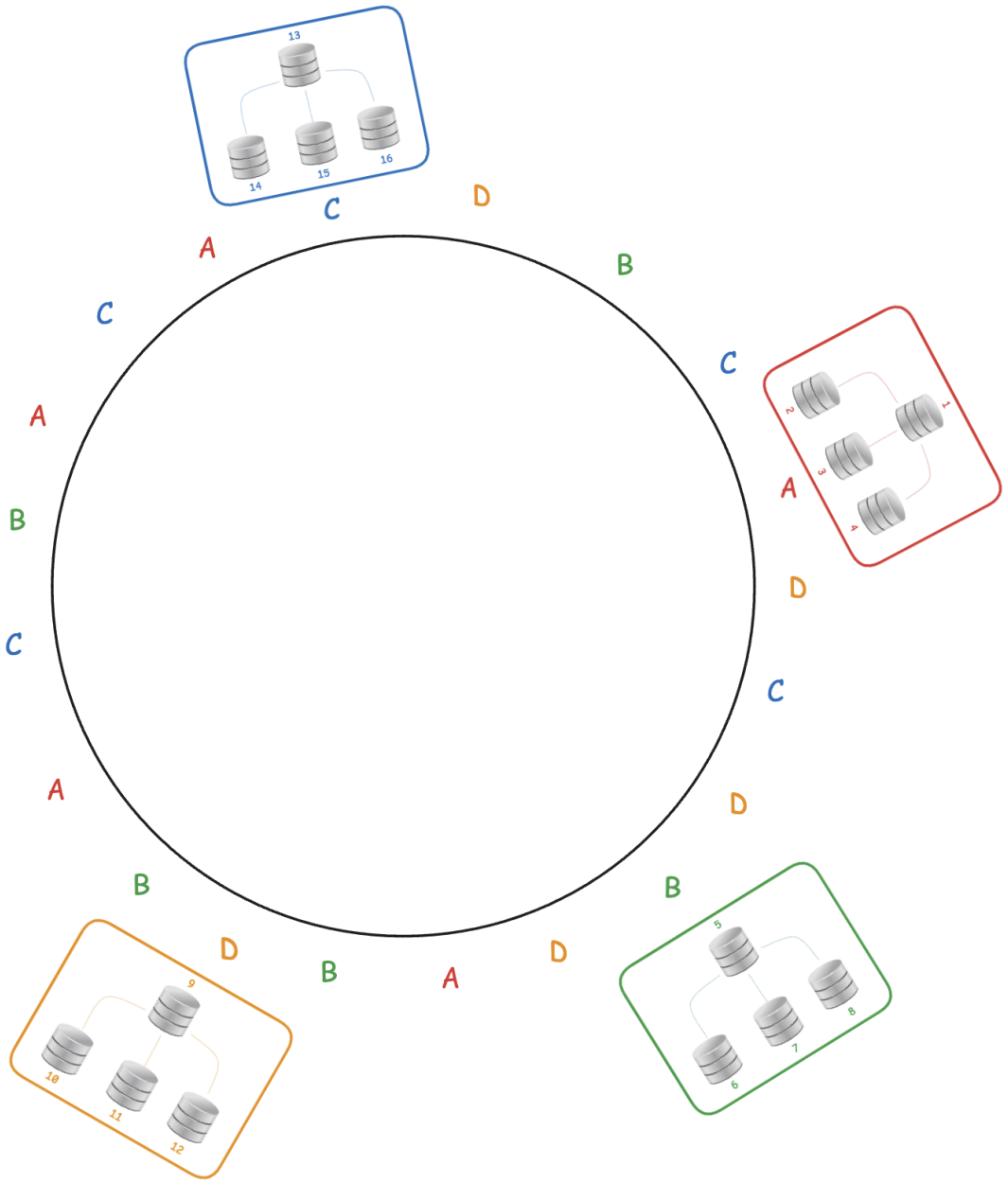
Q: What gets placed on the ring?

Servers (== shard) get placed on the ring

Q: Can a "shard" go down?

Yes. A shard (==server) can go down.
In this case, the request of the users originally going to that server will automatically get routed to the next server in the ring.
How does the data reach the next server in the ring? Son Pari

Sharding + Master-Slave Replication



Q: Shard vs Server

1 Shard = X servers
(X is the replication factor)

Each shard is comprised of multiple servers in a master-slave configuration.
All the servers within the same shard contain the same data (replication)

Q: What gets placed on the ring?

Shards are placed on the ring *(not the servers)*

Q: Can a "shard" go down?

No. A server within a shard can crash. However, and entire shard cannot crash (highly unlikely).
Because a shard is multiple servers. For a shard to crash, all servers within the shard must crash together (unlikely)

Q: What happens when a server crashes?

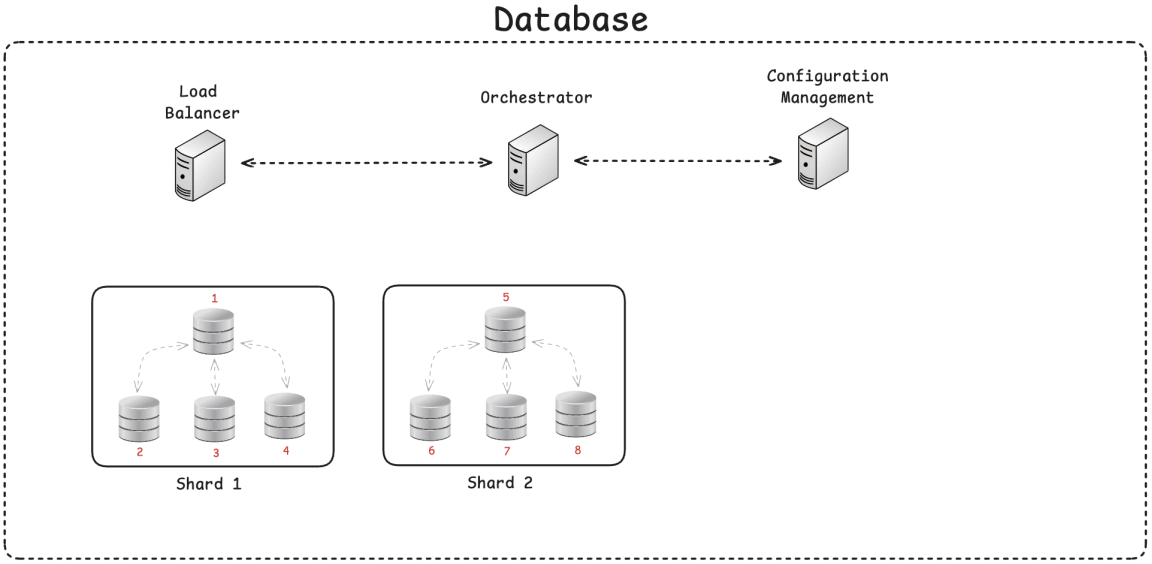
No worries.

- If the master within a particular shard crashes, then the slaves of that shard will re-elect a new master.
- If a slave within a particular shard crashes, then we can just read from some other slave.

The shard will continue functioning!

Database Orchestration

Orchestra



Replication Factor (X)

The *(minimum)* number of copies that must be maintained for any piece of data.
In the above diagram, X = 4

Configuration Management

If multiple servers need to coordinate to act as a singular database, then they need to maintain some common configuration.
For example

- What servers are available (health-check/heartbeat)
- What shards exist
    - What servers are assigned to each shard
    - Who's the master
    - Who are the slaves
- Replication factor
- Other
    - what dialect of SQL is being used
    - AWS key
    - ...

A distributed configuration management system

1. Maintains all configuration
2. Ensures that all servers always agree on the configuration of the system

Apache Zookeeper

Orchestrator

The brains behind the distributed database

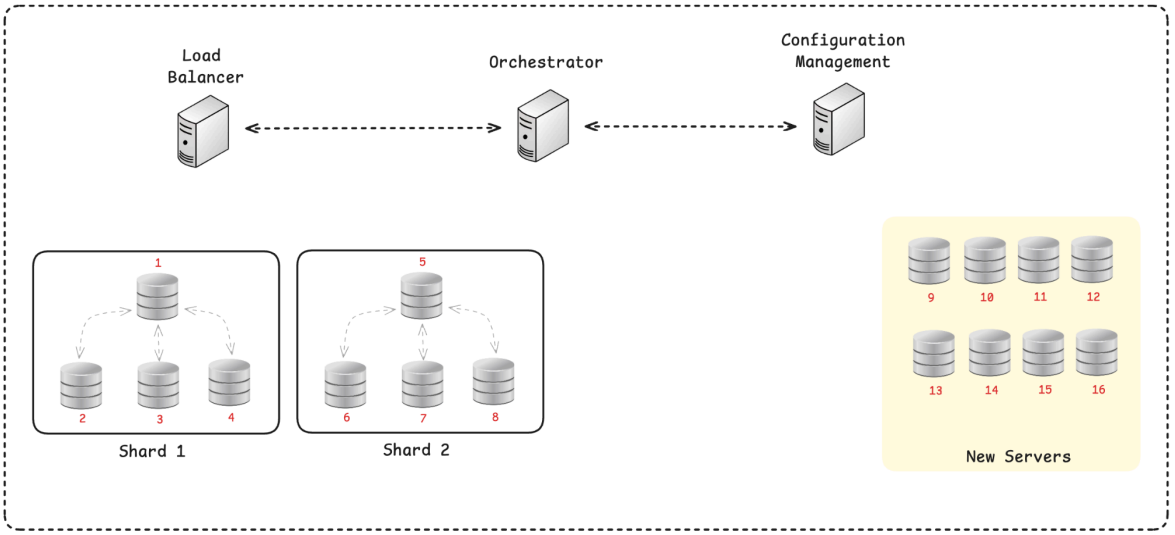
Orchestrator is responsible for ensuring

1. The replication factor must be maintained
    a. if a server crashes within a shard - the number of copies of data of that shard has decreased - we must add a new server within that shard (so the replication factor is maintained)
2. Deciding which servers belong to which shards
3. Adding new shards (scale out) / Removing existing shards (scale down)
4. Ensuring that the load is equally distributed across all servers & shards
    a. the routing happens via the LB
    b. Orchestrator will command the shards & servers to arrange

Adding New Servers

After some time, we realize that the database servers are close to their max capacity (*at 80% capacity*)

The human developers will purchase, configure & connect new servers, and they will inform the DB orchestrator about these new servers.



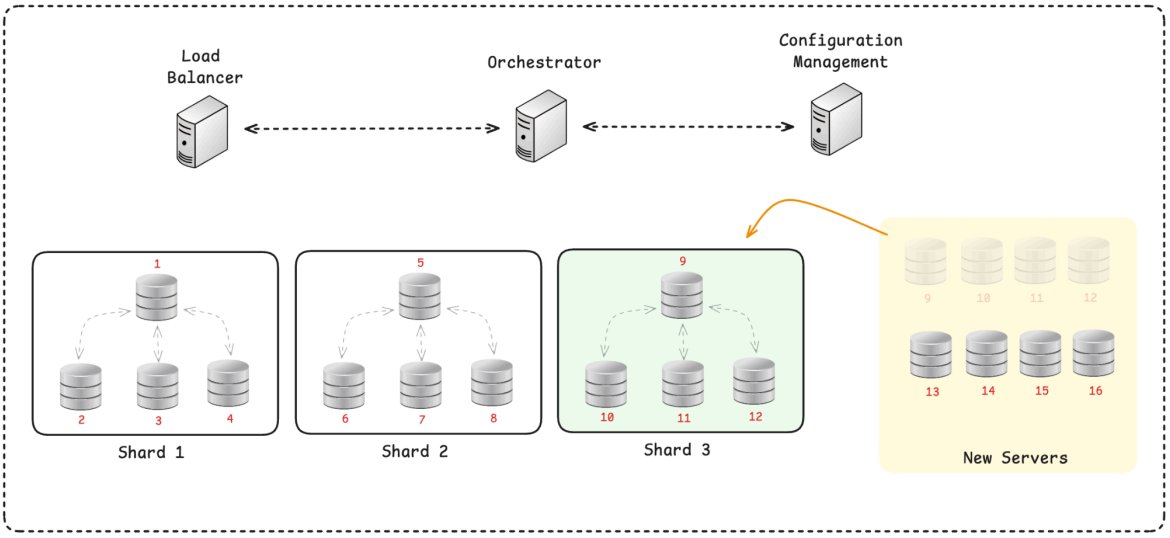
Q: What should the Orchestrator do with these new servers?

It assigns them to shards (existing or new)

Q: Should we assign these servers to an existing shard?

No. Each shard already has X servers (sufficient copies)
Assigning a server to an existing shard means we're increasing the replication factor (unnecessary – because we already have the desired replication factor)

We should assign these servers to a new shard ⇒ **Create new shards using these servers,** so that our existing shards can give away some of their load to the new shard



Q: How many new shards can we create?

If the replication factor is `X (=4)`, and we just added `N (=8)` new servers, then we can create up to `floor(N / X)` new shards.

Q: Should we create new shards with all these new servers?

If we create new shards from all available servers, what happens when a server crashes?

- The replication factor for the shard will decrease *(become less than the desired replication of X)*
- To restore the replication factor, we must add a new server to that shard to replace the crashed server
    - but we don't have any new servers - we already used all of them

So no, we should keep some servers reserved for emergency situations.

Q: When a new shard is created, how do we distribute the data to this new shard?

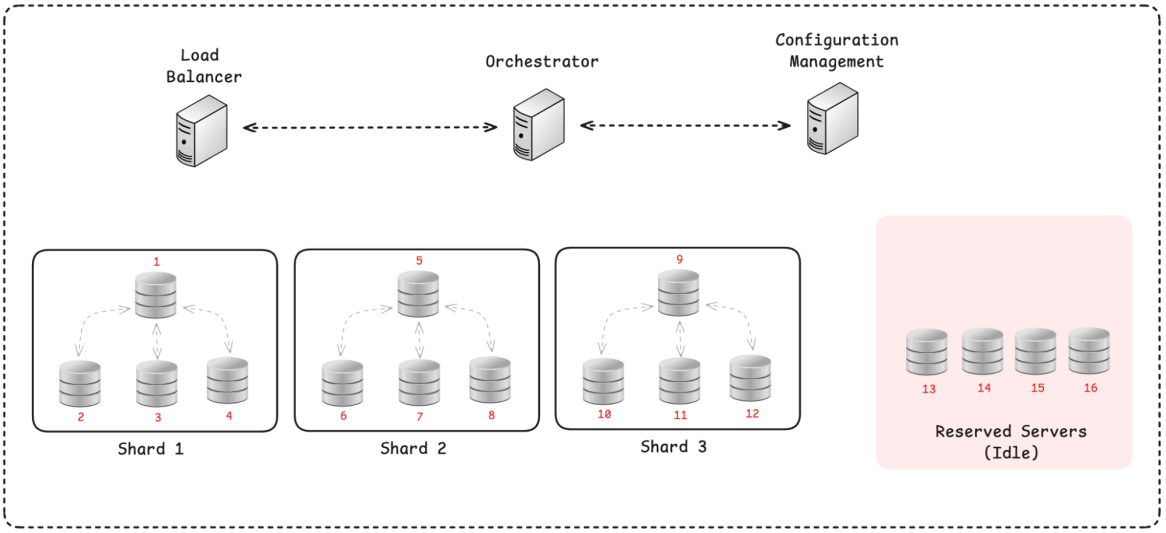
We need to ensure that we move the data with as little database downtime as possible: *Seamless shard creation - Simulation & Real phase*

Q: When a crashed server recovers, what happens?

We treat the crashed server as a new server

Reserved Servers

We should always maintain some of the servers in reserved, just in case, so that if any server server crashes within a shard, we can replace it with one of the reserved servers.



Q: What's the optimal number of reserved servers to maintain?

~~Number of reserved servers == Number of shards~~
~~*(1 reserved server for each shard)*~~

~~Number of reserved servers = X = Replication Factor~~

`P(server i is down on a given day) = p_crash`
Suppose we know that a typical server crashes every 1 year *(from past experience, or manufacturer contract guidelines, or expert advice, …)*
It takes 2 days to repair the crashed server.
`p_crash` = (2 days / 365 days) = 0.5%

Given **`N`** total servers, how many servers would be down (in expectation) on any given day?
`E (number of down servers on any given day) = p_crash * N`
= E = 0.5% * 1000
= 5 servers
Basically, if we have 1000 servers, and a typical server crashes once per year and takes 2 days to repair, then on any given day, we expect around 5 servers to be down on avg.

So should we maintain 5 reserved servers at all times?
No!

- half the days, the number of crashed servers would be < 5
  - so we have sufficient reservers
- but half of the days, the number of crashed servers would be > 5
  - insufficient reserves!

We should ensure that we (almost) always have sufficient reserves

`P(the number of crashed servers on any given day <= R) > 99%`
What's the minimum value of R (reserved servers), such that with 99% probability, we should have sufficient reserved servers for any given day.

Q: Should we keep these reserved servers idle?

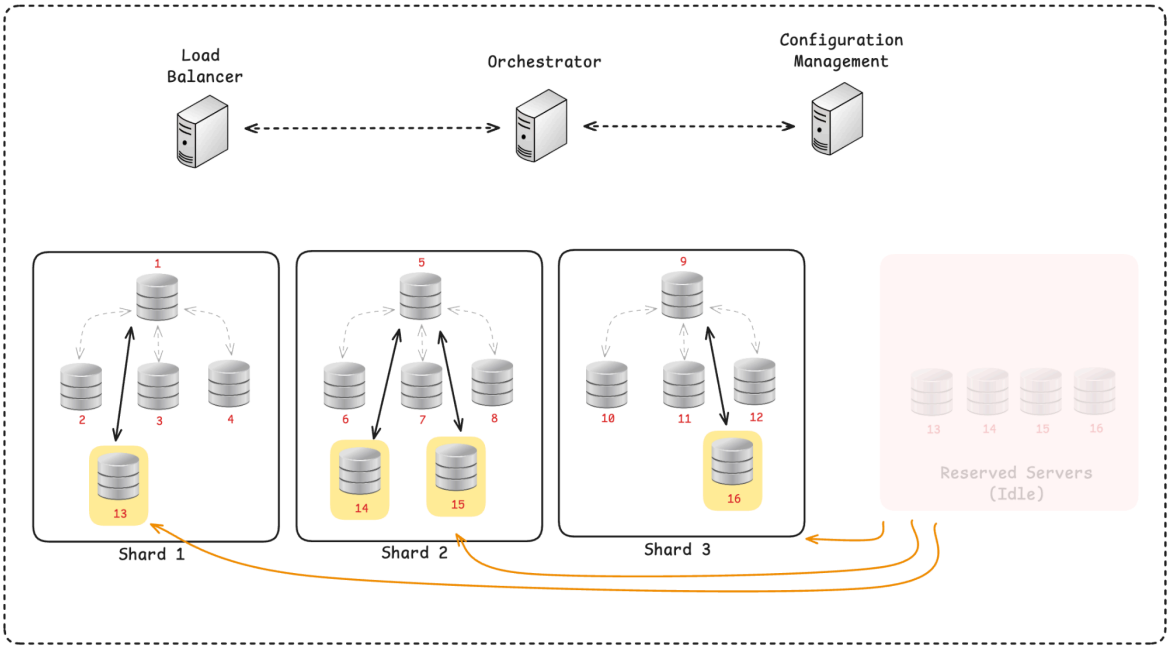
No. Servers are expensive.
We want to ensure that all the servers are being used somehow at all times. Minimize the wastage.

Q: What should we do with these reserved servers then?

We should not create a new shard ⇒ because then, we will not have any reserved servers.

Use these reserved servers as **extra "read replicas"** in existing shards.

1. They will not become masters - they will remain as slaves
2. They will have a marker on them that says that they're a reserved server



Q: Is it okay to have extra read replicas?

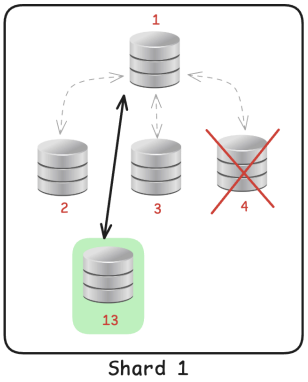
It is not necessary to have extra replicas - because already have sufficient replicas (X) in each shard.
But if we do have extra read replicas, that's even better – faster reads (because now this new replica will handle some of the read load)

Note: the replication factor (X) is a global configuration. That doesn't change. That's still `X=4`

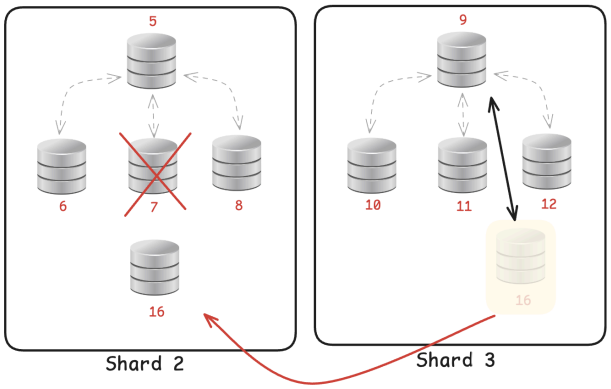
Replication factor just tells you the *min number of replicas that you need to maintain. You're allowed to exceed that, as long as you ensure that you're not wasting resources.

Q: What happens if a server crashes within a shard?

- If the shard already has extra read replicas (reserved servers) - just mark the reserved server as not-reserved.



- If the shard does not have extra read replicas, we will snatch away a reserved server from some other shard to replace the crashed server *(delete any existing data & replace it with data of the current shard)*



Note: *throughout all this, when a server has crashed, there's no downtime for any user! Because only a server has crashed - the shard still has multiple servers where the reads/writes can happen.*

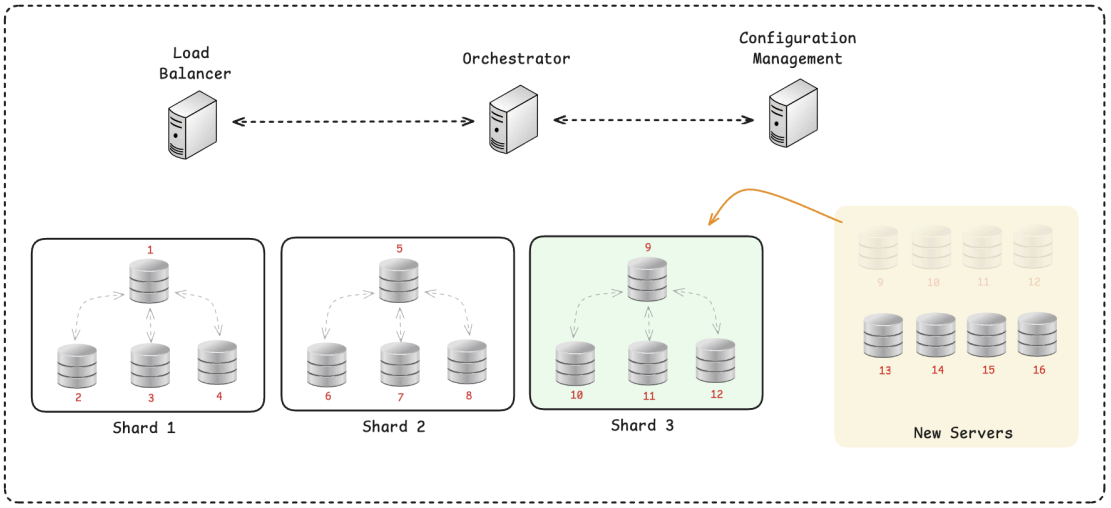
Seamless Shard Creation

When the orchestrator decides to create a new shard out of the new servers, how does the data movement happen?

1. The new shard must take away some data from every existing shard
    a. reduce the load of each existing shard equally

2. There should be minimal downtime for the users that are getting migrated to this new shard

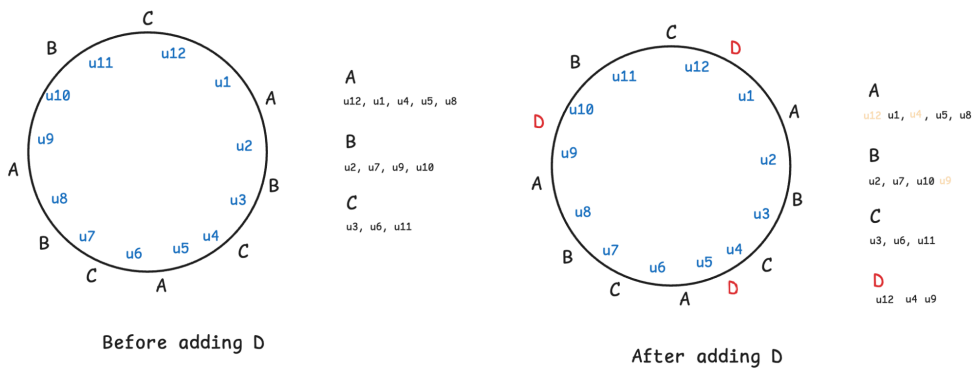
1. Simulation Phase



Database Orchestrator decides to create a new shard.
It doesn't tell anyone. It will keep this idea to itself, and it will first of all run a simulation.

Orchestrator runs a simulation

1. Orchestrator knows everything about the database (via the config management system)
    a. what users exist
    b. what servers exist
    c. what hashes are we using for consistent hashing
2. Orchestrator will run a simulation
    a. **without informing the Load Balancer** / any other server
    b. what happens in the consistent hashing ring when this new shard gets added
    c. it figures out which ***users will be moved to this new shard***?
    d. in the below diagram, ABCD are shards



    e. it will initiate a data copy from the existing shards to the new shard for this set of users

```
for each existing shards
    select *
    from table1, table2, ..
    where user_id in (4, 9, 12, ..)

insert this data into the new shard
```

Assume that this data copying got initiated at time instance t = t0
After some time the data copying will finish - let's say this happens at **t = t1**

During this time (between t0 ... t1) new data by these users was still being created in the existing shards

- because the orchestrator was running the simulation & data copying without informing the LB
- the LB was still sending the requests to the existing shards
- LB is not aware of the new shard that is being created

For the users that were being moved *(u4, u9, u12)*, some new data will get created in the existing shards during this time - **Delta**

Q: Won't the data copy process automatically also copy the new changes?
No!
Databases have a cursor — if you add new data while you're iterating over the existing data with a cursor, the cursor will NOT see the new data at all! (Isolation)

This means that at **t = t1**, the original data of these users *(u4, u9, u12)* has been copied to the new shard, but they still have some extra data (Delta) that has not been copied *(because it was created while the original data was being copied).*
Delta is missing!

2. Real Phase

Begins at t = t1 after the initial data copy has finished (but the delta is missing)

1. Orchestrator announces to the LB about the new shard
2. The LB will start sending requests to the new shard for these set of users
3. But, there's still some data (*Delta*) missing from this new shard
    - this means that if these moved users *(u4, u9, u12)* try to look at their data, their requests will go to the new shard, but, they will discover that some data is missing!
4. Orchestrator will initiate copying of the the delta to the new shard

```
for each existing shards
    select *
    from table1, table2, ..
    where user_id in (4, 9, 12, ..)
          and modified_at >= (t0)
```

```
insert this data into the new shard
```

Note: *the orchestrator is not actually running queries for the delta copying. In reality, it uses the WAL file to do this delta copying.*
The original data is copied using queries, but the delta is copied using the WAL file.

Consistency while creating a new Shard

Imagine that the delta copy completes at **t = t2**

What happens during the time b/w t1 and t2 when the majority of the data has been copied, but the delta is missing from the new shard?

During the Real Phase (t1 → t2), while the delta is being copied, and the LB is aware of the new shard, which means that is forwarding requests to the new shard, even through the new shard has missing data!
What should the shard do with these requests?

Serve the requests (read/write)

Since the delta is still being copied, there is a consistency issue. (No downtime)
Eventual consistency.
b/w t1 & t2, the users that are being moved might feel that some of their data is missing. But eventually (at t2) all of their data will once again be available & consistent.

Deny the requests (read/write)

Until the delta copying is completed, the new shard must deny any incoming requests. (Downtime)
Once the Real Phase has completed (delta copying has finished at t2), only then, we can resume all the read & write requests.
Low availability (for some time for some users).

Why are we doing this 2 step process?

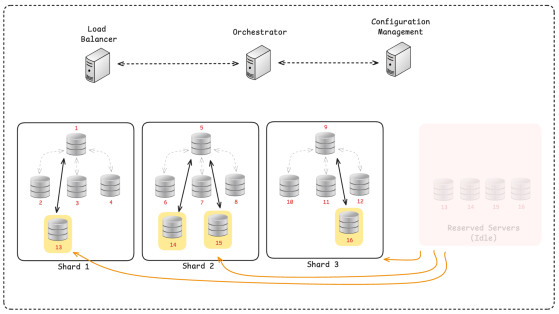
- **1 step process**
  - the new shard needs to copy 1 TB of data to itself. For immediate consistency, we have to wait for a long time (1 hour) for this data to be fully copied
  - long downtime
- **2 step process**
  - downtime happens only during real phase (delta copying)
  - total data for new shard is large (1TB), the delta is small
    - delta is the data created while the original data is being copied to the new shard (1 hour timeframe)
    - delta will be pretty small (a few 100 MBs max)

- copying the delta will not take a long time (a few seconds max)
- effectively reduced downtime from 1 hour to a few seconds (for immediate consistency)
    - for eventual consistency, there's no downtime

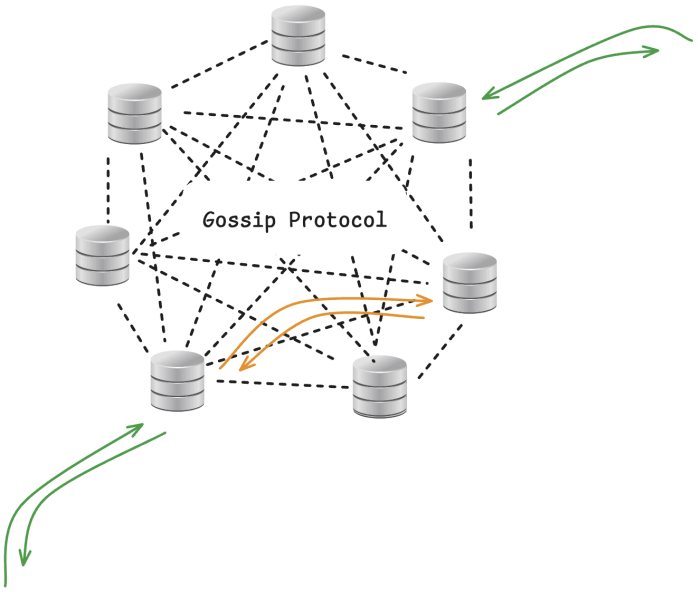
Sharding + Multi-Master Replication

Multi-master removes the distinction b/w master & slave.
Every server acts on its own & handles both reads & writes.
Every server acts as a master (handles writes) and as a slave (handles reads, and syncs up with others)

- In Master-Slave, there's many types (heterogenity) of servers
    - load balancer
    - orchestrator
    - configuration management
    - master
    - slave
    - reserved slave



- In Multi-master, there's just 1 type of server
    - each server can act on its own, and can handle everything
    - each server can perform routing (LB), each server is aware of other servers (orchestrator), each server handles writes (master), handles reads (slave)
    - all servers communicate with each other *(via Gossip protocol)*

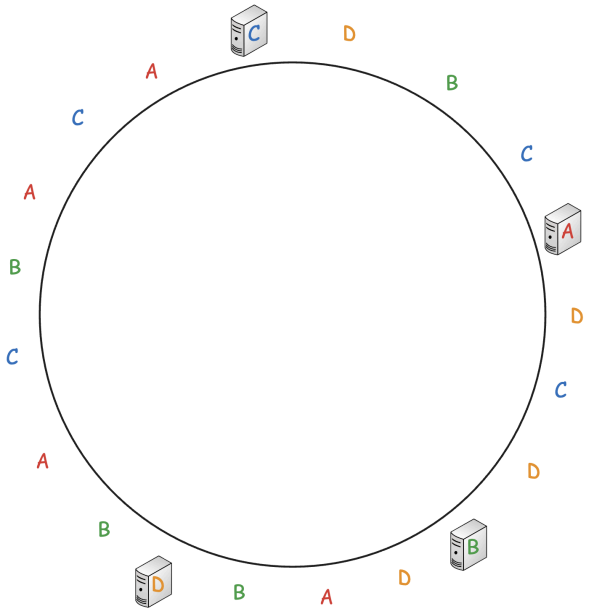


Q: Shard vs Server

Difference b/w shard & server no longer exist.
Each server acts as an overlapping database - it holds multiple partitions.
Because each server has to perform sharding (holds some subset of data) & replication (holds multiple distinct subsets of data)

Q: What gets placed on the ring?

Place servers on the ring



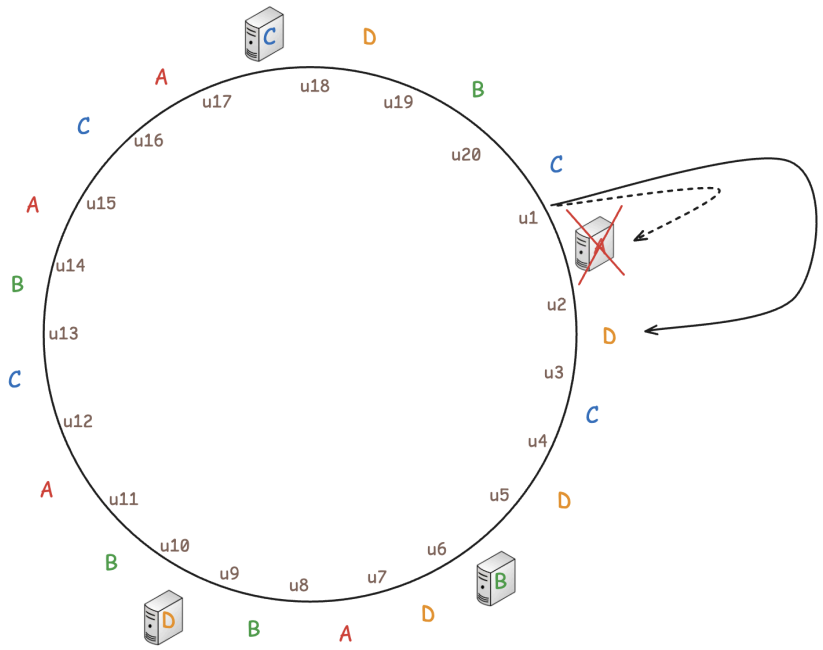
Q: Can a "shard" go down?

There's no concept of shards now.

Sharding still exists, but sharding & replication has been merged together.

Q: What happens when a server crashes?

The requests for all the users earlier routing to the crashed server will automatically get redirected to other servers (via consistent hashing)



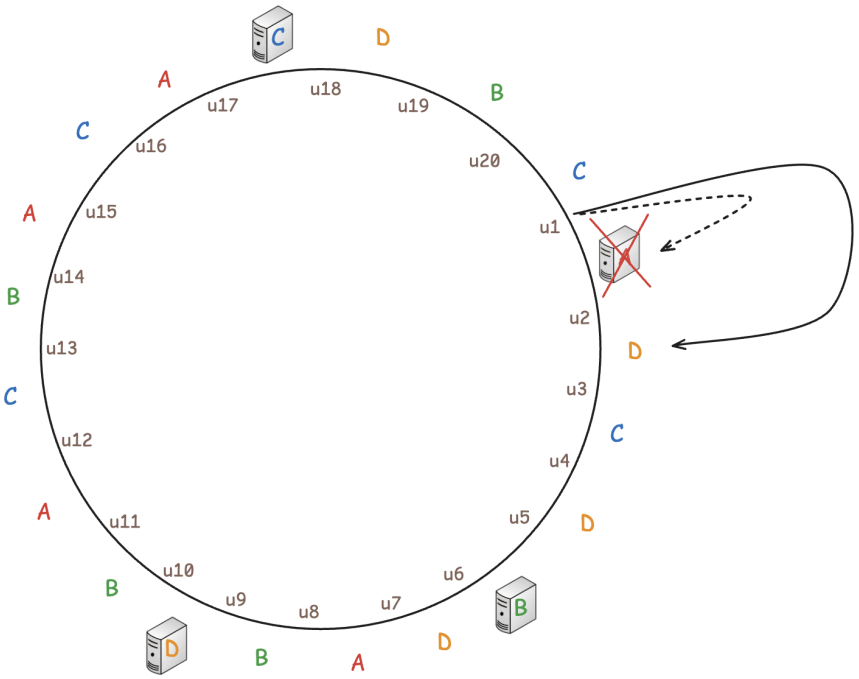
Q: How does the user's data arrive at the other servers?

Since we're doing replication along with sharding - each piece of data must be written in multiple places (at least 2 writes atomically, other writes happen async)

Q: What sort of consistency does multi-master provide?

We use Tunable Consistency `(X, R, W)` with Multi-master

Q: Which servers should contain which user's replicas?



N = 4 Servers: { A, B, C, D }
K = 5 hashes *(each server gets 5 spots on the ring)*
Replication Factor: 3 (each piece of data should have at least 3 copies)
Users: {1 .. 20}

Server A

Primary (1): {u1, u7, u11, u14, u16}
Replica (2): {u20, u6, u10, u13, u15}
Replica (3): {u19, u5, u9, u12, u14}

Server B

Primary (1): {u5, u8, u10, u13, u19}
Replica (2): {u4, u7, u9, u12, u18}
Replica (3): {u3, u6, u8, u11, u17}

Server C

Primary (1): {u3, u12, u15, u17, u20}
Replica (2): {u2, u11, u14, u16, u19}
Replica (3): {u1, u10, u13, u15, u18}

Server D

Primary (1): {u2, u4, u6, u9, u18}

Replica (2): `{u1, u3, u5, u8, u17}`
Replica (3): `{u20, u2, u4, u7, u16}`

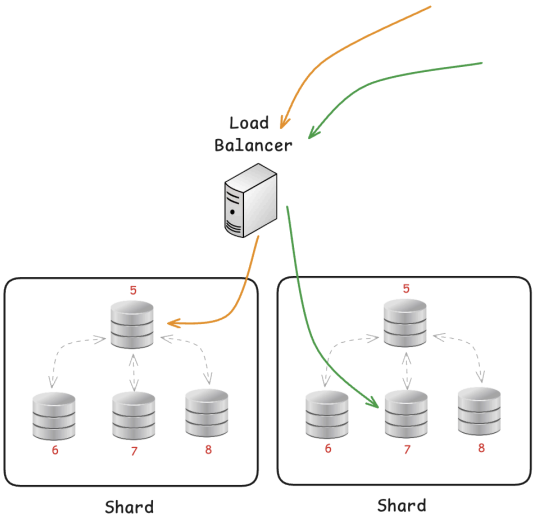
Note how u1's data is replicated across 3 different servers, since the replication factor (X) is 3. Similarly, u12's data can be found replicated across 3 different servers.

Note: in our example, A stores both the primary (1) and secondary (3) for u14.
But in practice, this will not happen (very unlikely) because the number of servers will be larger

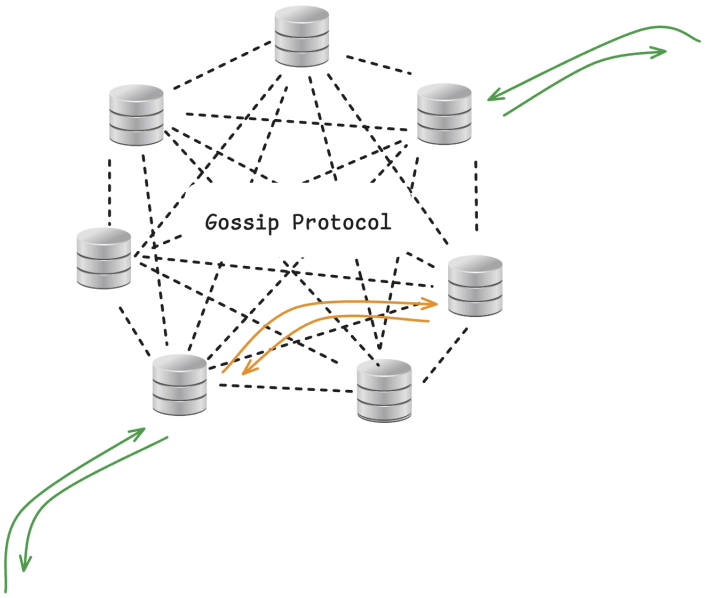
We never want to have multiple copies of the same user's data in the same server - if the server crashes, we will lose multiple replicas.
(multiple replicas for same data in 1 server defeats the purpose of replication)

Load Balancing & Routing

In Master-Slave, the LB is responsible for routing the request to the correct server (master or slave). Servers are heterogenous (therefore, more difficult to manage)
All requests come to the LB first.



In Multi-Master, any request can come to any server (higher availability in general)
The server will perform consistent hashing & figure out which server(s) contain the data to handle this request, and the server will forward the request to the appropriate server.
Servers are homogenous (very easy to manage)



Each server behaves as the LB, Orchestrator, Master, Slave, ...

Servers communicate with each other continuously via the Gossip Protocol (https://en.wikipedia.org/wiki/Gossip_protocol)

All Abstractions are Leaky

No matter how well-built & how powerful & robust an abstraction is, it will break down under certain circumstances.

Abstractions come at a cost. It is very rare to have 0 cost abstractions.
Especially when you're trying to optimize performance, you want to break abstractions.

For example - Hotstar uses AWS, and AWS supports auto-scaling. However, AWS auto-scaling takes up to 30 mins to react, whereas hotstar needs to react within minutes (during live matches) - so Hotstar had to build their own auto-scaling logic.

Therefore, even though using a *Managed Database Service* like AWS RDS will abstract away all the database internals from you and provide you features like sharding & replication, it is important to understand how these things work internally.